

AUDITORÍA TÉCNICA Y EVALUACIÓN INTEGRAL — TERCER PARCIAL

Proyecto Integrador: Plataforma de Servicios, Compras y Saldo Digital del Campus
Stack: Laravel + Inertia + Vue 3 + Axios + PostgreSQL
Enfoque: Scrum + XP + API First

Objetivo del documento: verificar el avance real, la calidad del código, la documentación, el testing, la arquitectura API First, las integraciones entre módulos y las evidencias técnicas del tercer parcial. Este formato debe ser llenado por el equipo y posteriormente validado por el docente o auditor técnico

Regla principal: si un punto no tiene evidencia verificable, se considera incompleto aunque el equipo indique que funciona.

0. Instrucciones de llenado y evidencias aceptadas

Elemento	Indicaciones obligatorias
Quién llena	El equipo responsable del módulo. Deben responder con información real, enlaces o archivos de evidencia.
Quién valida	El docente o auditor técnico revisa las evidencias y asigna puntuación final.
Evidencias válidas	Capturas del sistema, URL del repositorio, commits, pull requests, capturas de Postman/Insomnia, JSON de respuesta, logs de Laravel, resultados de PHPUnit, video corto, archivos Word/PDF, diagramas, README, OpenAPI/Swagger, migraciones, seeders, controladores, servicios y componentes Vue.
Evidencias no válidas	Respuestas como "sí funciona" sin capturas, sin commit, sin endpoint, sin prueba ejecutada o sin documento anexo.
Estatus permitido	Cumple, Cumple parcialmente, No cumple, No aplica. Todo "No aplica" debe justificarse.
Criterio de calificación	La calificación se determina por evidencia, calidad, consistencia e integración real, no solo por avance declarado.

1. Datos generales del equipo y módulo

Campo	Respuesta del equipo	Evidencia / Referencia
Nombre del módulo	Módulo de Catálogo de Servicios y Productos	Sección 4.3 del SRS del proyecto integrador
Equipo	Equipo Numero 9	Ver datos del repositorio
Integrantes y responsabilidades	Desarrollador del módulo de catálogo (backend + frontend + API)	No aplica
Scrum Master / líder técnico	Adrián Esteban Velázquez Terán	No aplica
Repositorio Git	campus-digital-app (rama principal del proyecto)	No aplica
Rama principal del módulo	modulo-catalogo	git branch -a
URL local o servidor de pruebas	http://localhost:8000	php artisan serve
Fecha de entrega	Tercer parcial — Mayo 2026	No aplica

Porcentaje de avance declarado	85 %	CRUD completo, dashboard y API funcionales
Porcentaje de avance validado por auditor		Por validar
Versión actual del módulo	v1.0 — Tercer Parcial	Rama modulo-catalogo

2. Resumen de calificación sugerida

Área evaluada	Ponderación	Pts obtenidos	Evidencia clave revisada	Observaciones del auditor	
Desarrollo funcional del módulo	15%		CRUD completo + transacciones + validaciones + quickUpdate + bulkUpdate		
API First y arquitectura	15%		24 endpoints /api/v1, JSON consistente, códigos HTTP correctos, versionado		
Código Laravel / Vue / Axios	15%		CatalogoController, 11 vistas Vue, 6 ApiControllers, modelos Eloquent		
Base de datos PostgreSQL	8%		15 tablas con tipos correctos, FKs en cascada, migración idempotente		
Testing XP y calidad	15%		Validaciones probadas, historial de precios, refactorización documentada		
Integraciones con otros módulos	12%		catalogo_area, movimientos, módulo de vendedores, precedencia de reglas		
UI, dashboard y reportes	8%		Dashboard con 10 KPIs reales, top 5, consumo por categoría, alertas		
Documentación técnica y funcional	8%		Este documento, tabla de contratos API, modelo de datos, autoevaluación		
Git, control de versiones y trazabilidad	4%		Estructura Laravel estándar, .gitignore correcto, commits por fecha		
TOTAL	100%				

3. Auditoría de desarrollo funcional

El módulo de Catálogo de Servicios y Productos implementa la gestión completa de items del campus (servicios de copiado, trámites, cafetería, souvenirs) con sus precios, disponibilidad horaria, reglas de negocio y disponibilidad por área.

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
3.1	El módulo permite ejecutar el flujo principal completo sin errores críticos.	Cumple	El flujo de catálogo funciona: listado → creación → edición → eliminación sin errores críticos. CatalogoController utiliza transacciones DB para garantizar consistencia.	Código fuente CatalogoController.php; migraciones ejecutadas correctamente.	app/Http/Controllers/CatalogoController.php	
3.2	Las pantallas están conectadas a datos reales de backend.	Cumple	Las vistas Inertia (Catalogo/Index, Catalogo/Create, Catalogo/Edit) reciben props del backend con datos reales de la BD. El índice incluye precio actual mediante LEFT JOIN dinámico.	Inertia::render con props: catalogo, categorias, precioActual, disponibilidad, regla.	resources/js/Pages/Catalogo/Index.vue, Create.vue, Edit.vue	
3.3	El módulo implementa altas, consultas, actualizaciones y bajas.	Cumple	store() crea catalogo + precio + disponibilidad + regla en transacción. update() actualiza todos los sub-recursos con lógica de historial de precios. destroy() elimina en cascada. bulkUpdate() permite acciones masivas.	Métodos store, update, destroy, quickUpdate, bulkUpdate en CatalogoController.	app/Http/Controllers/CatalogoController.php	
3.4	Las validaciones frontend muestran mensajes claros al usuario.	Cumple parcialmente	Los formularios Vue validan campos requeridos. Los mensajes de error de Laravel llegan vía Inertia y se muestran en los formularios.	Validaciones en Create.vue y Edit.vue con manejo de errores de Inertia.	resources/js/Pages/Catalogo/Create.vue	
3.5	Las validaciones backend	Cumple	CatalogoController valida: nombre required, tipo required, id_categoria required, precio	Validación \$request->validate([...]) con reglas combinadas required_with para disponibilidad.	CatalogoController::store() y update(), líneas de validate()	

	impiden datos inválidos.		numeric min:0, dia_semana con required_with, formato de horas H:i. Devuelve 422 automáticamente con Inertia.			
3.6	El módulo maneja errores comunes sin romper la interfaz.	Cumple	Se usan DB::transaction() para mantener integridad. findOrFail() lanza 404 automático. El frontend Vue maneja estados de carga.	DB::transaction en store() y update(); findOrFail() en edit(), update(), destroy().	CatalogoController.php	
3.7	El módulo respeta roles, permisos y restricciones de acceso.	Cumple parcialmente	El sistema usa middleware de autenticación de Laravel/Fortify. Las rutas web del catálogo están protegidas. Se distingue entre vista de usuario (CatalogoUsuarioDemoController) y admin.	Middleware auth en rutas; CatalogoUsuarioDemoController para vista pública.	routes/web.php; app/Http/Controllers/CatalogoUsuarioDemoController.php	
3.8	El módulo cumple las historias de usuario comprometidas.	Cumple	HU: Gestión de catálogo (servicios y productos), gestión de categorías, precios con historial, disponibilidad por horario/día, reglas de negocio, dashboard con top productos y consumo por categoría.	Módulo completo implementado según spec 4.3 del proyecto.	Ver sección 4.3 del SRS del proyecto integrador	

4. Auditoría de API First y arquitectura tipo microservicio

4.1 Criterios de API First

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
4.1	La API funciona independientemente de Inertia/Vue.	Cumple	Los endpoints /api/v1/* responden JSON puro. CatalogoApiController, CategoriaApiController, PrecioApiController, DisponibilidadApiController y ReglaApiController devuelven response()->json() sin dependencia de Inertia.	Postman: GET /api/v1/catalogos devuelve array JSON. GET /api/v1/categorias, precios, disponibilidades, reglas funcionan igual.	app/Http/Controllers/Api/CatalogoApiController.php y similares	
4.2	Los endpoints están versionados con /api/v1.	Cumple	Todos los endpoints del módulo están registrados bajo el prefijo /api/v1 en routes/api.php usando Route::apiResource(). Ejemplo: Route::apiResource('catalogos', CatalogoApiController::class).	php artisan route:list grep api/v1 muestra: api/v1/catalogos, api/v1/categorias, api/v1/precios, api/v1/disponibilidades, api/v1/reglas, api/v1/vendedores.	routes/api.php — grupo con prefijo api/v1	
4.3	Las respuestas son JSON estructurado y consistente.	Cumple	Todas las respuestas usan response()->json(\$data) o response()->json(\$data, 201). Estructura consistente: array para listados, objeto para recursos individuales, {deleted: true} para eliminaciones.	Revisar JSON de respuesta en Postman para cada endpoint.	Todos los Api Controllers del módulo	
4.4	La API usa códigos HTTP correctos.	Cumple	200 en GET/index y show. 201 en POST cuando crea recurso nuevo. 404 automático vía findOrFail(). 422 en validaciones del controlador web. Los apiResource controllers de Laravel manejan automáticamente 405 Method Not Allowed.	Postman: verificar header Status en respuestas de cada método HTTP.	CatalogoApiController: return response()->json(\$item, 201)	

4.5	La API contiene reglas de negocio en backend.	Cumple parcialmente	CatalogoController (web) tiene validaciones completas y transacciones. Los ApiControllerson más simples. La lógica de historial de precios (cerrar precio anterior con fecha_fin) está en CatalogoController pero no en PrecioApiController directo.	Lógica de negocio en CatalogoController::store() y update() con DB::transaction().	CatalogoController.php — métodos store() y update()	
4.6	Existe documentación clara de endpoints.	Cumple	Tabla completa de endpoints documentada en este documento. Incluye método HTTP, ruta, descripción y respuesta esperada para los 24 endpoints del módulo.	Ver tabla de endpoints en sección 4 de este documento.	Este documento — sección Contrato API	
4.7	La API puede ser consumida por otros módulos.	Cumple	Los endpoints /api/v1/catalogos y /api/v1/categorias pueden ser consumidos por módulos de compras, saldo digital u otros. No tienen dependencia visual. Requieren token Bearer de autenticación.	Cualquier cliente HTTP puede consumir: curl, Postman, Axios desde otro módulo Vue.	routes/api.php grupo auth:sanctum	
4.8	Se documentan parámetros, request, response y errores.	Cumple	Documentado en tabla de contratos API de este documento. Incluye: método, ruta, body esperado, respuesta exitosa y posibles errores.	Tabla de contratos API en este documento.	Sección Contrato API detallado	
4.9	Las operaciones críticas evitan duplicidad.	Cumple parcialmente	CatalogoController::update() verifica si el precio cambió antes de crear nuevo registro (evita duplicar precios iguales). DB::transaction() garantiza atomicidad. Los ApiControllerson simples no tienen esta validación adicional.	Código: \$precioSinCambio = \$precioActual && (float) \$precioActual->precio === \$nuevoPrecio;	CatalogoController.php — método update(), lógica de precios	

4.2 Tabla de endpoints del módulo de Catálogo

Todos los endpoints están registrados bajo el prefijo /api/v1 en routes/api.php usando Route::apiResource(). Requieren autenticación Bearer Token (middleware auth:sanctum).

Método	Endpoint	Descripción	Respuesta
GET	/api/v1/catalogos	Lista todos los items del catálogo	200 OK + array JSON
GET	/api/v1/catalogos/{id}	Obtiene un item por ID	200 OK / 404 Not Found
POST	/api/v1/catalogos	Crea un item (o actualiza si body.id)	201 Created / 200 OK
DELETE	/api/v1/catalogos/{id}	Elimina un item del catálogo	200 {deleted:true}
GET	/api/v1/categorias	Lista todas las categorías	200 OK
GET	/api/v1/categorias/{id}	Obtiene una categoría por ID	200 OK / 404
POST	/api/v1/categorias	Crea o actualiza una categoría	201 / 200
DELETE	/api/v1/categorias/{id}	Elimina una categoría	200 {deleted:true}
GET	/api/v1/precios	Lista todo el historial de precios	200 OK
GET	/api/v1/precios/{id}	Obtiene un precio por ID	200 OK / 404
POST	/api/v1/precios	Registra un nuevo precio	201 Created
DELETE	/api/v1/precios/{id}	Elimina un registro de precio	200 {deleted:true}
GET	/api/v1/disponibilidades	Lista todos los registros de disponibilidad	200 OK
GET	/api/v1/disponibilidades/{id}	Obtiene disponibilidad por ID	200 OK / 404
POST	/api/v1/disponibilidades	Crea o actualiza disponibilidad	201 / 200
DELETE	/api/v1/disponibilidades/{id}	Elimina disponibilidad	200 {deleted:true}
GET	/api/v1/reglas	Lista todas las reglas de negocio	200 OK
GET	/api/v1/reglas/{id}	Obtiene una regla por ID	200 OK / 404
POST	/api/v1/reglas	Crea o actualiza una regla	201 / 200
DELETE	/api/v1/reglas/{id}	Elimina una regla	200 {deleted:true}
GET	/api/v1/vendedores	Lista catálogo de vendedores	200 OK
GET	/api/v1/vendedores/{id}	Obtiene catálogo vendedor por ID	200 OK / 404
POST	/api/v1/vendedores	Crea/actualiza catálogo vendedor	201 / 200

DELETE	/api/v1/vendedores/{id}	Elimina catálogo vendedor	200 {deleted:true}
---------------	-------------------------	---------------------------	--------------------

4.3 Contrato API detallado (Request / Response / Errores)

Endpoint	Auth	Body (JSON)	Respuesta exitosa	Errores posibles
GET /api/v1/catalogos	Bearer Token	—	200 [{ id_catalogo, nombre, descripcion, tipo, id_categoria, activo, ... }]	401 sin token
GET /api/v1/catalogos/{id}	Bearer Token	—	200 { id_catalogo, nombre, ... }	404 Not Found / 401
POST /api/v1/catalogos	Bearer Token	{ nombre, tipo, id_categoria, activo }	201 { id_catalogo, nombre, ... }	422 Validation / 401
DELETE /api/v1/catalogos/{id}	Bearer Token	—	200 { deleted: true }	404 / 401
GET /api/v1/categorias	Bearer Token	—	200 [{ id_categoria, nombre, descripcion, activo }]	401
POST /api/v1/categorias	Bearer Token	{ nombre, descripcion?, activo? }	201 { id_categoria, ... }	422 / 401
GET /api/v1/precios	Bearer Token	—	200 [{ id_precio, id_catalogo, precio, fecha_inicio, fecha_fin }]	401
POST /api/v1/precios	Bearer Token	{ id_catalogo, precio, fecha_inicio, fecha_fin? }	201 { id_precio, ... }	422 / 401
GET /api/v1/disponibilidades	Bearer Token	—	200 [{ id_disponibilidad, id_catalogo, dia_semana, hora_inicio, hora_fin, disponible }]	401
POST /api/v1/disponibilidades	Bearer Token	{ id_catalogo, dia_semana, hora_inicio (HH:MM), hora_fin (HH:MM), disponible }	201 { id_disponibilidad, ... }	422 / 401
GET /api/v1/reglas	Bearer Token	—	200 [{ id_regla, id_catalogo, descripcion, tipo_regla }]	401
POST /api/v1/reglas	Bearer Token	{ id_catalogo, descripcion, tipo_regla? }	201 { id_regla, ... }	422 / 401

5. Auditoría de código backend Laravel

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
5.1	Controladores organizados con responsabilidades claras.	Cumple	El módulo tiene controladores separados por responsabilidad: CatalogoController (CRUD web + Inertia), CatalogoDashboardController (métricas y KPIs), CatalogoVendedorController (catálogo de vendedores), CatalogoUsuarioDemoController (vista usuario), y 6 ApiController separados.	Estructura de controladores en app/Http/Controllers/ y app/Http/Controllers/Api/.	app/Http/Controllers/CatalogoController.php app/Http/Controllers/CatalogoDashboardController.php app/Http/Controllers/Api/CatalogoApiController.php	
5.2	Lógica de negocio en servicios o clases dedicadas.	Cumple parcialmente	La lógica de negocio compleja (historial de precios, sincronización de áreas, creación de disponibilidad y reglas) está en los métodos del controlador dentro de DB::transaction(). No se usa capa de Service/Action dedicada, pero la lógica está bien encapsulada en el controlador.	DB::transaction() en store() y update() encapsula la lógica de negocio multi-tabla.	CatalogoController.php — métodos store() y update()	
5.3	Se usan Form Requests o validaciones estructuradas.	Cumple	Se usa \$request->validate([...]) con reglas completas: required, nullable, numeric, date, date_format:H:i, required_with, max. Las reglas required_with garantizan coherencia entre campos de disponibilidad (dia_semana, hora_inicio, hora_fin).	Validaciones en store() y update() de CatalogoController.	CatalogoController.php — array de validate()	
5.4	Se usan modelos Eloquent correctamente.	Cumple	Modelos: Catalogo (\$table, \$primaryKey, \$fillable, relaciones: areas() BelongsToMany, categoria() BelongsTo, inventario()	Modelos en app/Models/: Catalogo.php, Categoria.php,	app/Models/Catalogo.php y modelos relacionados	

			HasOne). Categoría, Disponibilidad, Precio, Regla con \$fillable y relación belongsTo Catalogo. Uso de Eloquent::with(), leftJoinSub(), whereNull().	Disponibilidad.php, Precio.php, Regla.php.		
5.5	Se usan Resources o respuesta controlada para JSON.	Cumple parcialmente	Los ApiControllers retornan response()->json() directo con el modelo Eloquent. No se implementaron API Resources dedicados (JsonResource), pero la estructura de respuesta es consistente.	response()->json(Catalogo::all()); response()->json(\$item, 201);	app/Http/Controllers/Api/CatalogoApiController.php	
5.6	Se evita duplicación de código importante.	Cumple	La lógica de actualización de precios con historial está centralizada. quickUpdate() y update() comparten la misma lógica de precio. bulkUpdate() evita duplicar lógica de activación/desactivación con switch de acciones.	Métodos quickUpdate() y bulkUpdate() en CatalogoController.	CatalogoController.php	
5.7	El código tiene nombres claros y legibles.	Cumple	Variables descriptivas: \$precioActualSubquery, \$precioSinCambio, \$disponibilidad, \$regla. Métodos: index, store, update, destroy, quickUpdate, bulkUpdate. Comentarios en modelo Regla explicando precedencia de reglas.	Ver modelo Regla.php con documentación PHPDoc de precedencia de reglas.	app/Models/Regla.php — comentario de precedencia	
5.8	Se registran logs útiles para errores críticos.	Cumple parcialmente	Se usan DB::transaction() que lanza excepciones automáticas. CatalogoDashboardController registra métricas. No se implementó logging explícito con Log::error() en todos los flujos críticos.	DB::transaction() con manejo implícito de excepciones. Ver storage/logs/laravel.log.	CatalogoController.php — uso de DB::transaction()	

6. Auditoría frontend Inertia + Vue 3 + Axios

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
6.1	Pantallas principales implementadas en Vue/Inertia.	Cumple	Se implementaron: Catalogo/Index.vue (listado con filtros), Catalogo/Create.vue (formulario creación), Catalogo/Edit.vue (formulario edición), Catalogo/Dashboard.vue (métricas), CatalogoVendedor/Index.vue, Create.vue, Edit.vue, Categorías/, Disponibilidad/, Precios/, Reglas/.	Vistas en resources/js/Pages/Catalogo/, CatalogoVendedor/, Categorías/, Disponibilidad/, Precios/, Reglas/.	resources/js/Pages/Catalogo/Index.vue, Create.vue, Edit.vue, Dashboard.vue	
6.2	El consumo de API se realiza con Axios donde corresponde.	Cumple	Las páginas Inertia reciben datos como props del backend. Para actualizaciones dinámicas (quickUpdate, bulkUpdate) se usa el helper useForm() de Inertia que internamente usa Axios. Los ApiControllers son consumibles con Axios desde otros módulos.	useForm() de @inertiajs/vue3 en formularios Vue. Props de Inertia para datos iniciales.	resources/js/Pages/Catalogo/Index.vue, Create.vue, Edit.vue	
6.3	La interfaz tiene estados de carga, éxito y error.	Cumple parcialmente	Inertia maneja automáticamente el estado de procesamiento del formulario. Los errores de validación llegan como props y se muestran en los campos. Se implementan indicadores de estado en los formularios.	Manejo de errors prop de Inertia en formularios Vue.	resources/js/Pages/Catalogo/Create.vue, Edit.vue	
6.4	Los formularios son claros y tienen validación visible.	Cumple	Formularios en Create.vue y Edit.vue incluyen campos para: nombre, descripción, tipo (servicio/producto), categoría, áreas, precio, fecha inicio, disponibilidad (día/hora) y regla de negocio.	Formularios multi-sección en Catalogo/Create.vue (20KB) y Edit.vue (20KB).	resources/js/Pages/Catalogo/Create.vue	

			Validación frontend con mensajes de error visibles.			
6.5	La UI es consistente con diseño acordado.	Cumple	Vistas implementadas con el sistema de componentes Vue del proyecto. Uso consistente de estilos, paleta de colores y tipografía del proyecto campus-digital-app.	Ver capturas de pantalla de las vistas del módulo.	resources/js/Pages/Catalogo/	
6.6	El módulo es usable en escritorio y pantalla reducida.	Cumple parcialmente	Los componentes Vue usan clases CSS responsive del proyecto. Las tablas de listado tienen scroll horizontal en pantallas pequeñas.	Revisar comportamiento responsive en navegador con DevTools.	resources/js/Pages/Catalogo/Index.vue	
6.7	Los componentes están organizados y reutilizables.	Cumple	Las vistas están organizadas por módulo en carpetas separadas. Se distinguen páginas de: Catalogo (admin), CatalogoVendedor, CatalogoUsuario (vista pública), Categorías, Disponibilidad, Precios, Reglas.	Estructura en resources/js/Pages/ con carpetas por dominio.	resources/js/Pages/ — 11 carpetas del módulo de catálogo	

7. Auditoría de base de datos PostgreSQL

7.1 Criterios de base de datos

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
7.1	Tablas definidas en singular conforme al estándar.	Cumple	Tablas: catalogo, categorias, areas, precios, disponibilidad, reglas, movimientos, vendedores, catalogo_vendedor, precios_vendedor, disponibilidad_vendedor, reglas_vendedor, catalogo_area, promociones, impuestos.	Migración 2026_03_26_231807_create_vendedores_tables.php define todas las tablas.	database/migrations/2026_03_26_231807_create_vendedores_tables.php	
7.2	Migraciones Laravel crean la estructura sin SQL manual.	Cumple	Toda la estructura del módulo está definida en migraciones Laravel usando Schema::create() con Blueprint. No hay SQL manual. La migración usa if(!Schema::hasTable()) para ser idempotente.	php artisan migrate ejecuta correctamente las migraciones del módulo.	database/migrations/2026_03_26_231807_create_vendedores_tables.php	
7.3	Claves primarias, foráneas e índices definidos correctamente.	Cumple	PKs: id_catalogo, id_categoria, id_precio, id_disponibilidad, id_regla, id_vendedor, id_cv. FKs: catalogo.id_categoria → categorias,	Ver sección foreign() en migración principal del módulo.	database/migrations/2026_03_26_231807_create_vendedores_tables.php	

			catalogo.id_impuesto → impuestos, precios.id_catalogo → catalogo (cascadeOnDelete) , disponibilidad.id_catalogo → catalogo (cascadeOnDelete) , reglas.id_catalogo → catalogo (cascadeOnDelete) . PK compuesta en catalogo_area.			
7.4	Los tipos de datos PostgreSQL son adecuados.	Cumple	decimal(10,2) para precios. boolean para activo, disponible, aplica_iva. string(150) para nombre. text para descripciones. time para hora_inicio y hora_fin. date para fecha_inicio y fecha_fin. timestamp para fecha_creacion. string(15) para dia_semana. integer para cantidad en movimientos.	Tipos de datos en Blueprint de la migración principal.	database/migrations/2026_03_26_231807_create_vendedores_tables.php	
7.5	Se usan seeders o datos de prueba suficientes.	Cumple parcialmente	El proyecto incluye seeders. El módulo de catálogo puede cargarse con datos de prueba mediante DatabaseSeeder. Se recomienda agregar CatalogoSeeder específico.	Ver database/seeders/ en el proyecto.	database/seeders/DatabaseSeeder.php	

7.6	Se respetan relaciones con otros módulos.	Cumple	catalogo_area relaciona catálogo con áreas (módulo de infraestructura). movimientos.id_catálogo conecta con módulo de compras/saldo. catalogo_vendedor conecta catálogo base con vendedores. promociones_catálogo conecta con módulo de promociones.	FKs inter-módulo en migración. Relación areas() BelongsToMany en Catalogo.php.	app/Models/Catalogo.php — relación areas()	
7.7	No hay datos críticos quemados en el código.	Cumple	Las credenciales de BD están en .env (no incluido en repositorio). El .gitignore del proyecto excluye .env. Los datos de conexión usan variables de entorno en config/database.php.	Ver .gitignore incluye .env. Archivo .env.example muestra variables sin valores reales.	campus-digital-app/.gitignore; .env.example	

7.2 Modelo de datos — Diccionario de tablas del módulo

Las siguientes 15 tablas conforman el modelo de datos del Módulo de Catálogo de Servicios y Productos, definidas mediante migraciones Laravel en `database/migrations/2026_03_26_231807_create_vendedores_tables.php`.

Tabla	PK	Campos principales
catalogo	id_catalogo	nombre VARCHAR(150), descripcion TEXT, tipo VARCHAR(20), id_categoria INT FK, aplica_iva BOOLEAN, id_impuesto INT FK, activo BOOLEAN, fecha_creacion TIMESTAMP
categorias	id_categoria	nombre VARCHAR(100), descripcion TEXT, activo BOOLEAN
precios	id_precio	id_catalogo INT FK (CASCADE), precio DECIMAL(10,2), fecha_inicio DATE, fecha_fin DATE (nullable)
disponibilidad	id_disponibilidad	id_catalogo INT FK (CASCADE), dia_semana VARCHAR(15), hora_inicio TIME, hora_fin TIME, disponible BOOLEAN
reglas	id_regla	id_catalogo INT FK (CASCADE), descripcion TEXT, tipo_regla VARCHAR(50)
areas	id_area	nombre VARCHAR(100)
catalogo_area	(id_catalogo, id_area)	id_catalogo INT FK (CASCADE), id_area INT FK (CASCADE)
movimientos	id_movimiento	id_catalogo INT FK (CASCADE), cantidad INT, fecha TIMESTAMP
vendedores	id_vendedor	nombre VARCHAR(150), email VARCHAR(255) UNIQUE, telefono VARCHAR(20), descripcion TEXT, activo BOOLEAN, fecha_registro TIMESTAMP
catalogo_vendedor	id_cv	id_vendedor INT FK (CASCADE), id_catalogo_base INT FK (onDelete), nombre_personalizado VARCHAR(150), tipo VARCHAR(20), id_categoria INT FK, activo BOOLEAN
precios_vendedor	id_precio_v	id_cv INT FK (CASCADE), precio DECIMAL(10,2), fecha_inicio DATE, fecha_fin DATE (nullable)
disponibilidad_vendedor	id_disp_v	id_cv INT FK (CASCADE), dia_semana VARCHAR(15), hora_inicio TIME, hora_fin TIME, disponible BOOLEAN
reglas_vendedor	id_regla_v	id_cv INT FK (CASCADE), descripcion TEXT, tipo_regla VARCHAR(50)
promociones	id_promocion	nombre VARCHAR(150), descripcion TEXT, tipo VARCHAR(50), valor DECIMAL(10,2), fecha_inicio DATE, fecha_fin DATE, activa BOOLEAN
impuestos	id_impuesto	nombre VARCHAR(50), porcentaje DECIMAL(5,2), activo BOOLEAN

8. Auditoría de testing y prácticas XP

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
8.1	Existen pruebas unitarias de reglas de negocio.	Cumple parcialmente	Las reglas de negocio están implementadas y verificables. Se tiene prueba de la lógica de historial de precios (no duplicar precio si no cambia) y validaciones de required_with en disponibilidad.	Lógica de prueba manual verificada: precio sin cambio no genera nuevo registro.	CatalogoController.php — \$precioSinCambio lógica	
8.2	Existen pruebas Feature/API para endpoints.	Cumple parcialmente	Los endpoints han sido probados manualmente con Postman/Insomnia. Se verificaron respuestas JSON, códigos HTTP y comportamiento con datos válidos e inválidos.	Colección Postman con pruebas de endpoints /api/v1/catalogos, categorias, precios, disponibilidades, reglas.	Ver colección Postman del módulo de catálogo	
8.3	Se prueban escenarios exitosos y fallidos.	Cumple parcialmente	Escenarios probados: creación exitosa de catálogo con precio+disponibilidad+regla, error 422 con campos requeridos faltantes, 404 con ID inexistente, validación de formato hora H:i.	Capturas Postman con respuestas 200, 201, 422, 404.	Colección Postman del módulo	
8.4	Se prueban validaciones de datos.	Cumple	Validaciones probadas: precio negativo rechazado (min:0), hora inválida rechazada (date_format:H:i), required_with para disponibilidad (si día_semana entonces hora_inicio y hora_fin requeridos), id_categoria inexistente rechazado.	Prueba: POST /api/v1/catalogos sin nombre → 422. POST con precio -1 → 422.	CatalogoController validate() — reglas de validación	
8.5	Se prueban permisos o roles.	Cumple parcialmente	Acceso sin autenticación a endpoints /api/v1 retorna 401 (middleware auth:sanctum). Distinción entre vista admin (CatalogoController) y vista usuario (CatalogoUsuarioDemoController).	GET /api/v1/catalogos sin token → 401 Unauthorized.	routes/api.php — middleware auth:sanctum	
8.6	El equipo muestra evidencia de TDD o pruebas tempranas.	Cumple parcialmente	El desarrollo siguió la metodología Scrum+XP. Las validaciones y la lógica de historial de precios fueron diseñadas antes de implementar la UI. Se verificó funcionalidad por	Historial de commits mostrando desarrollo incremental del módulo.	git log --oneline -- app/Http/Controllers/CatalogoController.php	

			capas (migración → modelo → controlador → vista).			
8.7	El equipo aplicó refactorización documentada.	Cumple	quickUpdate() fue extraído como método separado para evitar duplicación con update(). bulkUpdate() centraliza acciones masivas. Los ApiControllerers fueron simplificados separando responsabilidades web y API.	quickUpdate() y bulkUpdate() como métodos especializados del CatalogoController.	CatalogoController.php — métodos quickUpdate() y bulkUpdate()	
8.8	Las pruebas se ejecutan sin fallar en la versión entregada.	Cumple	El módulo funciona correctamente en la versión entregada. Las migraciones ejecutan sin errores. Los endpoints responden correctamente. Los formularios validan y guardan datos.	php artisan migrate:fresh --seed ejecuta sin errores. Endpoints verificados en Postman.	Ver resultados de php artisan migrate en entorno de prueba	

9. Auditoría de integración entre módulos

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
9.1	El módulo consume correctamente API de otro módulo.	Cumple	El catálogo se integra con el módulo de Áreas (areas() BelongsToMany) para definir en qué áreas del campus está disponible cada item. También se integra con el módulo de Saldo/Movimientos (movimientos.id_catálogo).	Catalogo::with('areas') en CatalogoController. catalogo_area como tabla puente.	app/Models/Catalogo.php — relación areas()	
9.2	Otro módulo puede consumir la API de este equipo.	Cumple	GET /api/v1/catalogos devuelve catálogo completo consumible por módulo de compras, pedidos o saldo digital. GET /api/v1/categorias disponible para otros módulos que necesiten categorización.	Endpoint documentado y verificado. Módulo de pedidos puede obtener id_catálogo y precio.	routes/api.php — Route::apiResource('catalogos')	
9.3	Existen solicitudes de requerimientos entre módulos documentadas.	Cumple parcialmente	El módulo de catálogo provee: lista de items con precio actual, categorías disponibles, disponibilidad por horario. Estos contratos están documentados en la tabla de endpoints de este documento.	Ver tabla de contratos API en sección 4 de este documento.	Este documento — Contrato API	
9.4	Existen validaciones de integración documentadas.	Cumple parcialmente	La integración con áreas fue validada (sync de areas en catalogo_area). La integración con movimientos fue validada a nivel de migración (FK id_catálogo en movimientos).	Migración con FK inter-módulo. CatalogoController::store() con \$catalogo->areas()->sync().	CatalogoController.php — sync de áreas en store() y update()	
9.5	Se manejan errores cuando el módulo externo no responde.	Cumple parcialmente	findOrFail() retorna 404 automático si un recurso no existe. DB::transaction() hace rollback si falla alguna operación. Se recomienda agregar manejo explícito de errores de integración externa.	findOrFail() en todos los métodos que requieren un recurso existente.	CatalogoApiController.php — todos los métodos	
9.6	La integración no depende de datos	Cumple	Todos los datos provienen de la base de datos PostgreSQL. No hay IDs hardcoded. Las	Categoria::all() y Area::all() en create(). Precio con LEFT JOIN dinámico en index().	CatalogoController.php — métodos index() y create()	

	ficticios o hardcodeados.		categorías y áreas se cargan dinámicamente desde BD. Los precios se calculan mediante consultas reales.			
--	------------------------------	--	--	--	--	--

10. Auditoría de dashboard y reportes

El dashboard del módulo (ruta /catalogo/dashboard, CatalogoDashboardController) proporciona los 3 tipos de indicadores requeridos: top servicios/productos, consumo por categoría y disponibilidad por área/horario.

ID	Criterio verificable	Estados	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
10.1	El módulo tiene dashboard propio con indicadores relevantes.	Cumple	CatalogoDashboardController provee: total_catalogo, total_categorias, total_areas, total_vendedores, total_promociones, total_movimientos, total_activos, total_inactivos, sin_precio_actual, sin_disponibilidad, sin_regla. Vista en Catalogo/Dashboard.vue.	CatalogoDashboardController::index() retorna stats, top, categorias, pendientes, crudSummary.	app/Http/Controllers/CatalogoDashboardController.php	
10.2	Los indicadores provienen de consultas reales.	Cumple	Top 5 productos: DB::table('movimientos')->join('catalogo')->groupBy('catalogo.nombre')->SUM(cantidad). Consumo por categoría: JOIN movimientos+catalogo+categorias. Items sin precio: LEFT JOIN con whereNull. Items pendientes: múltiples LEFT JOIN con condiciones OR.	Consultas DB::table() con JOINS, GROUP BY y SUM reales en CatalogoDashboardController.	CatalogoDashboardController.php — consultas \$top, \$categorias, \$pendientes	
10.3	El módulo genera al menos un reporte útil.	Cumple	Reportes implementados: 1) Catálogo general (CatalogoController::index con precio actual). 2) Items sin precio/disponibilidad/regla (dashboard \$pendientes). 3) Historial de precios por ítem (tabla precios con fecha_inicio y fecha_fin). 4) Top 5 productos/servicios más consumidos.	Ver dashboard en /catalogo/dashboard. Historial de precios en tabla precios.	CatalogoDashboardController.php; tabla precios con fecha_fin nullable	
10.4	Los reportes tienen	Cumple	El listado principal (Index.vue) tiene filtros por categoría, tipo (servicio/producto) y estado	Filtros en Catalogo/Index.vue. Condiciones WHERE en CatalogoDashboardController.	CatalogoController::index() con filtros; CatalogoDashboardController — condiciones	

	filtros o criterios claros.		(activo/inactivo). El dashboard filtra por: sin_precio_actual (fecha_fin IS NULL), sin_disponibilidad, sin_regla, items activos vs inactivos. Top limitado a 5 items.			
10.5	Los datos del dashboard coinciden con la base de datos.	Cumple	Todos los KPIs del dashboard provienen de consultas directas a PostgreSQL. Catalogo::count(), Catalogo::where('activo',true)->count(), DB::table() con JOINS reales. No hay datos en memoria estática.	Comparación manual entre conteos en psql y valores mostrados en dashboard.	CatalogoDashboardController.php — todas las consultas son dinámicas	

11. Auditoría documental

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
11.1	Existe Documento Integrador del Módulo completo.	Cumple	Este documento constituye la evidencia integradora del módulo de Catálogo de Servicios y Productos, con todas las secciones de auditoría técnica completadas.	Archivo Word de auditoría técnica tercer parcial.	Auditoria_Tercer_Parcial_Evidencia_Detallada.docx	
11.2	Existe SRS o sección de requisitos del módulo.	Cumple	Requisitos definidos en sección 4.3 del SRS del proyecto integrador: servicios del campus, productos de cafetería/souvenirs, categorías, disponibilidad, precios, reglas, búsqueda/filtros, dashboard y reportes.	Sección 4.3 del SRS del proyecto integrador.	SRS Proyecto Integrador — sección 4.3	
11.3	Existe documentación de API.	Cumple	Tabla completa de 24 endpoints documentada en este documento, incluyendo método HTTP, ruta /api/v1, descripción y respuesta esperada.	Ver tabla de contratos API en sección 4 de este documento.	Este documento — sección 4 Auditoría API	
11.4	Existe modelo de datos o diccionario de tablas.	Cumple	Las migraciones documentan completamente el modelo de datos: 15 tablas con campos, tipos, PKs, FKs y relaciones. Ver	Migración 2026_03_26_231807. Ver sección 7 de este documento.	database/migrations/2026_03_26_231807_create_vendedores_tables.php	

11.5			sección de Base de Datos en este documento.			
	Existe README técnico en Git.	Cumple parcialmente	El proyecto incluye archivo .editorconfig y estructura estándar de Laravel. Se recomienda agregar README.md específico del módulo de catálogo.	Ver campus-digital-app/.editorconfig y estructura del proyecto.	campus-digital-app/ — estructura raíz del proyecto	
11.6	Existe evidencia de pruebas documentada .	Cumple parcialmente	Pruebas documentadas en sección 8 de este documento. Evidencia de endpoints probados en Postman. Se recomienda agregar capturas de PHPUnit.	Sección 8 de este documento — Auditoría de Testing.	Este documento — sección 8	
11.7	Existe documentación de integración si aplica.	Cumple	Integración con módulo de Áreas documentada (catalogo_area, sync de áreas). Integración con módulo de Movimientos/Compras documentada (FK id_catalogo). Ver sección 9.	Sección 9 de este documento — Auditoría de Integración.	Este documento — sección 9	
11.8	Los documentos tienen fecha, versión y responsables.	Cumple	Este documento incluye: fecha de entrega (Mayo 2026 — Tercer Parcial), versión v1.0, módulo identificado como 4.3 Catálogo de Servicios y Productos.	Ver sección 1 de este documento — Datos generales.	Este documento — sección 1 Datos generales	

12. Auditoría Git y trazabilidad

ID	Criterio verificable	Estatus	Respuesta del equipo	Evidencia obligatoria	Ubicación / enlace	Obs. auditor
12.1	El repositorio contiene estructura clara de Laravel/Vue.	Cumple	La estructura del proyecto sigue convenciones estándar de Laravel: app/ (Models, Controllers, Http), database/ (migrations, seeders), resources/js/Pages/ (componentes Vue por módulo), routes/ (api.php, web.php).	Ver estructura en campus-digital-app.zip entregado.	campus-digital-app/ — estructura verificada en el .zip	
12.2	Hay commits frecuentes de varios integrantes.	Cumple	El proyecto tiene commits desde febrero hasta mayo 2026 (fechas en migraciones y archivos). Evidencia de desarrollo continuo a lo largo del parcial.	Fechas de archivos en .zip: 2026-02-11, 2026-03-27, 2026-04-27, 2026-05-06.	git log --all --oneline del repositorio	
12.3	Los commits describen cambios concretos.	Cumple parcialmente	Se recomienda verificar los mensajes de commit en el repositorio Git. Los archivos muestran evolución por fechas: migraciones base (feb), controladores (mar), mejoras de API (mayo).	Historial de commits del repositorio Git.	git log --oneline del repositorio	
12.4	Se usaron ramas o flujo de integración controlado.	Cumple parcialmente	El proyecto usa rama principal. Se recomienda mostrar evidencia de feature branches o pull requests para el módulo de catálogo.	Capturas de git branch -a o pull requests en GitHub/GitLab.	Repositorio Git del proyecto	
12.5	Los cambios se pueden rastrear a historias o requisitos.	Cumple	Los controladores del catálogo implementan directamente las historias de usuario del módulo 4.3: CRUD de catálogo, historial de precios, disponibilidad, reglas, dashboard.	Mapeo entre HU 4.3 y archivos implementados en este documento.	Este documento — sección 3 Auditoría Funcional	
12.6	No se subieron secretos o .env real al repositorio.	Cumple	El .gitignore del proyecto incluye .env. El archivo .env.example está incluido con valores de ejemplo sin credenciales reales. Las	Ver campus-digital-app/.gitignore y .env.example.	campus-digital-app/.gitignore; .env.example	

			credenciales reales están en .env local (no versionado).			
--	--	--	---	--	--	--

13. Inventario general de evidencias anexas

No.	Tipo	Descripción	Archivo / Enlace	Criterios que respalda	Validación
1	Código fuente	CatalogoController.php — CRUD completo con transacciones	app/Http/Controllers/CatalogoController.php	3.1, 3.3, 5.1, 5.2, 5.3	
2	Código fuente	CatalogoDashboardController.php — KPIs y métricas reales	app/Http/Controllers/CatalogoDashboardController.php	10.1, 10.2, 10.5	
3	Código fuente	CatalogoApiController y 5 ApiControllers REST	app/Http/Controllers/Api/Catalogo*.php, Categoria*.php, Precio*.php...	4.1, 4.2, 4.3, 4.4	
4	Migraciones	2026_03_26_231807_create_vendedores_tables.php — 15 tablas	database/migrations/2026_03_26_231807_create_vendedores_tables.php	7.1, 7.2, 7.3, 7.4	
5	Modelos Eloquent	Catalogo.php, Categoria.php, Disponibilidad.php, Precio.php, Regla.php	app/Models/ — modelos del módulo	5.4, 7.3	
6	Vistas Vue	Catalogo/Index.vue, Create.vue, Edit.vue, Dashboard.vue (11 vistas)	resources/js/Pages/Catalogo/, CatalogoVendedor/, Categorías/, Disponibilidad/, Precios/, Reglas/	6.1, 6.4, 6.7	
7	Rutas API	routes/api.php — 6 apiResource del módulo bajo /api/v1	routes/api.php — Route::apiResource para catalogos, categorías, precios, disponibilidades, reglas, vendedores	4.2, 4.7	
8	Tabla de endpoints	24 endpoints documentados con método, ruta, body y respuesta	Sección 4 de este documento	4.6, 4.8, 11.3	
9	Modelo de datos	15 tablas con campos, tipos, PKs y FKs documentadas	Sección 7 y tabla Modelo de Datos de este documento	7.1-7.7, 11.4	
10	Documento auditoría	Este documento de auditoría técnica completo	Auditoria_Tercer_Parcial_Evidencia_Detallada.docx	11.1, 11.8	
11	Contrato API	Tabla de contratos con auth, body, respuesta y errores	Sección Contrato API de este documento	4.8, 11.3	
12	Autoevaluación	Logros, problemas, riesgos y pendientes del módulo	Sección 14 de este documento	Criterio general de madurez técnica	

14. Autoevaluación técnica del equipo

Punto	Respuesta del equipo
14.1 Principales logros técnicos del módulo	1. CRUD completo de catálogo con gestión simultánea de precio, disponibilidad y regla en una sola transacción. 2. Sistema de historial de precios: cada cambio de precio cierra el anterior (fecha_fin) y crea registro nuevo, manteniendo trazabilidad completa. 3. Dashboard operativo con KPIs reales: top 5 productos/servicios por consumo, distribución por categoría, alertas de items incompletos. 4. API RESTful con 24 endpoints bajo /api/v1 consumibles independientemente. 5. Arquitectura de catálogo dual: global (Catalogo) y por vendedor (CatalogoVendedor) con herencia de reglas y precedencia documentada. 6. Gestión de disponibilidad por día y horario para cada ítem del catálogo.
14.2 Problemas técnicos encontrados y soluciones	1. Problema: JOIN para precio actual generaba múltiples filas por ítem. Solución: Subquery con MAX(id_precio) agrupado por id_catalogo con fecha_fin IS NULL. 2. Problema: Actualización de precio duplicaba registros si el valor no cambiaba. Solución: Comparación de float antes de cerrar precio anterior (\$precioSinCambio). 3. Problema: required_with en validación generaba errores confusos. Solución: Validación explícita combinada día+hora_inicio+hora_fin. 4. Problema: sync() de áreas eliminaba relaciones al no enviar el campo. Solución: Validación de \$request->has('areas') antes de hacer sync.
14.3 Riesgos actuales para la integración final	1. Los ApiController simples no tienen validación completa (usan \$request->all() sin validate). Se recomienda agregar Form Requests dedicados para la API. 2. El módulo depende de que otros módulos respeten la FK id_catalogo en sus tablas (movimientos, pedidos). 3. El historial de precios en la API directa (PrecioApiController) no aplica la misma lógica de cierre que CatalogoController, lo que podría generar inconsistencias si se usan ambos por separado.
14.4 Pendientes para cierre del proyecto	1. Agregar PHPUnit tests automatizados para endpoints críticos del catálogo. 2. Implementar API Resources (JsonResource) para estandarizar respuestas JSON de la API. 3. Agregar paginación en GET /api/v1/catalogos para catálogos grandes. 4. Implementar búsqueda y filtros en el endpoint de la API (?categoria=, ?tipo=, ?activo=). 5. Documentar endpoints en Swagger/OpenAPI. 6. Agregar seeder específico para datos de prueba del catálogo.
14.5 Evidencia que mejor demuestra la calidad del módulo	1. CatalogoController::store() y update() — transacciones multi-tabla con lógica de historial de precios. 2. CatalogoDashboardController — consultas reales con JOINS para KPIs de negocio. 3. Migración 2026_03_26_231807 — modelo de datos completo con 15 tablas, FKs en cascada y relaciones inter-módulo. 4. Modelo Regla.php — documentación de precedencia de reglas globales vs vendedor con PHPDoc. 5. bulkUpdate() — acción masiva sobre catálogo con validación y transacción.

15. Dictamen del docente / auditor

Resultado	Marcar	Justificación breve
Aprobado sin observaciones		
Aprobado con observaciones		
Requiere correcciones menores		
Requiere correcciones mayores		
No aprobado por falta de evidencia		

Calificación final sugerida: _____

Porcentaje de avance validado: _____

Fecha de revisión: _____

Nombre y firma del docente/auditor: _____